

# BLUESTOCK FINTECH

Capstone Project I - Mutual Fund Analytics

## Mutual Fund Analytics Platform

End-to-End Data Engineering, ETL Pipeline & Interactive Dashboard

| Company      | Bluestock Fintech Pvt. Ltd.                           |
|--------------|-------------------------------------------------------|
| Domain       | Mutual Fund / Fintech                                 |
| Project Type | Individual Capstone I- Mutual Fund Analytics (1 Week) |
| Duration     | 7 Working Days   ~50-55 Hours                         |
| Data Source  | AMFI India (Public), mfapi.in, NSE/BSE Public Data    |
| Technologies | Python, SQL, Power BI / Tableau, Pandas, Matplotlib   |
| Submission   | PDF Report + Dashboard + GitHub Repository            |
| Prepared By  | Intern / Data Analyst — Bluestock Fintech             |
| Date         | September 2026                                        |

**40 Schemes**

Real Fund Schemes

**10 Datasets**

Provided

**87K+ Rows**

Transaction Data

**4.5 Yrs**

NAV History

# Table of Contents

---

|                                                        |    |
|--------------------------------------------------------|----|
| 1. Project Overview & Objective .....                  | 3  |
| 2. Problem Statement .....                             | 4  |
| 3. Data Sources & Datasets .....                       | 5  |
| 4. System Architecture & ETL Pipeline.....             | 7  |
| 5. 7-Day Task Breakdown .....                          | 9  |
| Day 1 Project Setup + Data Ingestion (ETL) .....       | 9  |
| Day 2 Data Cleaning + SQL Database Design .....        | 11 |
| Day 3 Exploratory Data Analysis (EDA) .....            | 13 |
| Day 4 Fund Performance Analytics .....                 | 15 |
| Day 5 Dashboard Development (Power BI / Tableau) ..... | 17 |
| Day 6 Advanced Analytics + Risk Metrics .....          | 19 |
| Day 7 Final Report + Presentation + Deployment .....   | 21 |
| 6. Technical Stack Details .....                       | 23 |
| 7. Deliverables & Evaluation Rubric .....              | 24 |
| 8. Appendix: Dataset Schema Reference.....             | 25 |

# 1. Project Overview & Objective

The objective of this project is to build an end-to-end Mutual Fund Analytics Platform using publicly available mutual fund data. The project focuses on collecting, cleaning, and transforming data through an ETL pipeline and storing it in a structured SQLite database.

It also aims to perform exploratory data analysis and evaluate mutual fund performance, risk, investor behaviour, SIPflows, and market trends. Finally, the project presents meaningful and actionable insights through an interactive PowerBI dashboard.

## 1.1 Business Context

The Indian mutual fund industry has grown at a remarkable pace. As of December 2025, the industry manages over **Rs. 81 lakh crore** in AUM across 1,908 schemes and **26.12 crore investor folios**. Monthly SIP inflows crossed an all-time high of **Rs. 31,002 crore** in December 2025, reflecting India's deepening equity culture. Bluestock Fintech wants a data platform that:

- Tracks NAV movements of 40+ key mutual fund schemes from top AMCs
- Monitors AUM growth trends for the 10 largest fund houses over 4+ years
- Analyses investor behaviour patterns across geographies and demographics
- Computes risk-adjusted return metrics (Sharpe, Sortino, Alpha, Beta)
- Benchmarks fund performance against Nifty 50, Nifty 100, BSE SmallCap indices
- Provides an executive-level interactive dashboard for fund selection

## 1.2 Project Objectives

| #  | Objective                                     | Outcome                       |
|----|-----------------------------------------------|-------------------------------|
| O1 | Build an ETL pipeline from raw AMFI data      | Automated Python script       |
| O2 | Design a normalised SQL schema for MF data    | 5-table star schema           |
| O3 | Perform comprehensive EDA on NAV & AUM data   | EDA notebook with charts      |
| O4 | Compute performance & risk metrics per scheme | Metrics dashboard             |
| O5 | Build an interactive BI dashboard             | Power BI / Tableau file       |
| O6 | Analyse investor transaction patterns         | Demographic insights          |
| O7 | Compare fund returns vs benchmark indices     | Alpha / tracking error report |
| O8 | Document and present the entire project       | PDF report + slides           |

## 2. Problem Statement

Despite the massive growth of India's mutual fund industry, individual investors and financial advisors often struggle to make data-driven fund selection decisions due to fragmented data, lack of unified analytics platforms, and complex financial metrics. This project solves real business problems:

### P1: Data Fragmentation

|                 |                                                                                                                                                                                                   |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Problem</b>  | NAV data, AUM data, SIP flow data and portfolio holdings are available on different sections of the AMFI website in different formats (TXT, PDF, HTML tables). No single unified database exists. |
| <b>Solution</b> | <b>Build an ETL pipeline that consolidates all data into a single SQLite/PostgreSQL database.</b>                                                                                                 |

### P2: Performance Comparison Gap

|                 |                                                                                                                                                                                            |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Problem</b>  | Investors cannot easily compare multiple funds across different AMCs on a risk-adjusted basis. Raw NAV data requires significant transformation to compute Sharpe ratio, Alpha, Beta, etc. |
| <b>Solution</b> | <b>Compute and visualise all key risk-return metrics in a single dashboard.</b>                                                                                                            |

### P3: No Benchmark Tracking

|                 |                                                                                                                                                                            |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Problem</b>  | Most retail investors do not know whether their fund is outperforming its benchmark index. Tracking error and information ratio require both NAV and benchmark index data. |
| <b>Solution</b> | <b>Join NAV data with benchmark index prices and compute rolling alpha and tracking error.</b>                                                                             |

### P4: Investor Behaviour Blind Spot

|                 |                                                                                                                                                            |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Problem</b>  | AMCs and distributors have limited visibility into how demographic and geographic factors influence SIP amounts, fund preferences and redemption patterns. |
| <b>Solution</b> | <b>Analyse investor transaction data to generate demographic segmentation and behavioural insights.</b>                                                    |

### P5: Slow Reporting

|                 |                                                                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Problem</b>  | Monthly MF reports are static PDFs that take days to prepare. Stakeholders need live, self-service dashboards with drill-down capability. |
| <b>Solution</b> | <b>Build a Power BI / Tableau dashboard refreshable from the ETL pipeline output.</b>                                                     |

### 3. Data Sources & Datasets

All datasets used in this project are derived from **publicly available, freely accessible sources**. No proprietary or confidential data is used. The primary sources are:

| Source             | URL / API                  | Data Type                | Update Freq.    |
|--------------------|----------------------------|--------------------------|-----------------|
| AMFI India         | www.amfiindia.com          | NAV, AUM, Folio, SIP     | Daily / Monthly |
| mfapi.in           | api.mfapi.in/mf/{code}     | Historical NAV (JSON)    | Daily           |
| mfddata.in         | mfddata.in/api/v1/schemes/ | NAV + Expense Ratio      | Daily           |
| NSE India          | nseindia.com/reports       | Benchmark Index Prices   | Daily           |
| BSE India          | bseindia.com               | BSE SmallCap Index       | Daily           |
| AMFI Monthly Notes | amfiindia.com/research     | Industry SIP & Flow Data | Monthly         |

#### 3.1 Dataset Inventory

|                                                                                                                        |                     |
|------------------------------------------------------------------------------------------------------------------------|---------------------|
| <b>01_fund_master.csv</b>                                                                                              | <b>40 rows</b>      |
| Master list of 40 real mutual fund schemes (AMFI codes, fund house, category, expense ratio, risk grade, fund manager) |                     |
| <b>02_nav_history.csv</b>                                                                                              | <b>~46,000 rows</b> |
| Daily NAV for all 40 schemes from Jan 2022 to May 2026, anchored to real AMFI NAV values from mfapi.in                 |                     |
| <b>03_aum_by_fund_house.csv</b>                                                                                        | <b>~90 rows</b>     |
| Quarterly AUM (in Rs. crore) for 10 fund houses from 2022 to 2025, sourced from real AMFI quarterly reports            |                     |
| <b>04_monthly_sip_inflows.csv</b>                                                                                      | <b>48 rows</b>      |
| Month-wise SIP inflow (Rs. crore), active SIP accounts, new registrations and SIP AUM — real AMFI Monthly Note data    |                     |
| <b>05_category_inflows.csv</b>                                                                                         | <b>~144 rows</b>    |
| Net inflows by fund category (Large Cap, Mid Cap, Small Cap, ELSS, Liquid, etc.) for FY 2024-25                        |                     |
| <b>06_industry_folio_count.csv</b>                                                                                     | <b>21 rows</b>      |
| Total mutual fund folios (in crore) broken by Equity, Debt, Hybrid — real AMFI published milestones                    |                     |
| <b>07_scheme_performance.csv</b>                                                                                       | <b>40 rows</b>      |

|                                                                                                              |                     |
|--------------------------------------------------------------------------------------------------------------|---------------------|
| 1yr/3yr/5yr returns, Sharpe, Sortino, Alpha, Beta, Max Drawdown, Std Dev — computed from NAV history         |                     |
| <b>08_investor_transactions.csv</b>                                                                          | <b>~32,000 rows</b> |
| Simulated SIP + Lumpsum + Redemption transactions for 5,000 investors across Indian states with demographics |                     |
| <b>09_portfolio_holdings.csv</b>                                                                             | <b>~320 rows</b>    |
| Top equity holdings (stock, weight %, sector) for equity mutual funds as of Dec 2025                         |                     |
| <b>10_benchmark_indices.csv</b>                                                                              | <b>~8,000 rows</b>  |
| Daily closing values for Nifty 50, Nifty 100, Nifty Midcap 150, BSE SmallCap, CRISIL Liquid & Gilt indices   |                     |

### 3.2 Key Real-World Data Points Embedded

| Metric                       | Value                                                  | Source                   |
|------------------------------|--------------------------------------------------------|--------------------------|
| SBI MF AUM Dec 2025          | Rs. 12.50 lakh crore (largest AMC in India)            | AMFI Quarterly           |
| ICICI Pru MF AUM Dec 2025    | Rs. 10.74 lakh crore                                   | AMFI Quarterly           |
| HDFC MF AUM Dec 2025         | Rs. 9.30 lakh crore                                    | AMFI Quarterly           |
| SIP Inflow Dec 2025          | Rs. 31,002 crore (all-time high at time of project)    | AMFI Monthly Note        |
| Active SIP Accounts Dec 2025 | 9.35 crore accounts                                    | AMFI Monthly Note        |
| Total MF Folios Dec 2025     | 26.12 crore                                            | AMFI / Business Standard |
| Industry AUM Dec 2025        | Rs. 81 lakh crore (Oct 2025 peak: Rs. 79.9 lakh crore) | AMFI                     |
| NAV anchor: HDFC Top 100     | Rs. 892.45 (Oct 2024) — from mfapi.in code 125497      | mfapi.in                 |

## 4. System Architecture & ETL Pipeline

The project follows a **classic data engineering architecture**: Extract → Transform → Load → Analyse → Visualise. This mirrors real-world fintech data pipelines used at companies like Zerodha, Groww, and Paytm Money.

### LAYER 1: DATA SOURCES (Extract)

- AMFI Daily NAV TXT (amfiindia.com/spages/NAVAll.txt)
- AMFI Historical NAV API (portal.amfiindia.com/DownloadNAVHistoryReport\_Po.aspx)
- mfapi.in REST API (GET /mf/{scheme\_code} — JSON, no auth required)
- AMFI Monthly Notes PDF (amfiindia.com/research-information/amfi-monthly)
- NSE/BSE Bhavcopy CSV (nseindia.com/reports)
- Pre-packaged CSV datasets provided with this project

### LAYER 2: DATA PROCESSING (Transform)

- Python (Pandas) — parse, clean, reshape, merge datasets
- Handle missing NAV values (weekends/holidays → forward-fill)
- Compute derived fields: daily returns, rolling averages, CAGR
- Normalise fund names (AMFI naming inconsistencies)
- Validate AMFI codes against master list
- Enrich transactions with fund metadata (join on amfi\_code)

### LAYER 3: DATA STORAGE (Load)

- SQLite (development) or PostgreSQL (production)
- 5-table star schema: dim\_fund, dim\_date, fact\_nav, fact\_transactions, fact\_performance
- Indexed on amfi\_code, date for fast query performance
- CSV flat-file backup for Tableau / Power BI direct import

### LAYER 4: ANALYTICS (Analy)

- Jupyter Notebooks for EDA (Matplotlib, Seaborn, Plotly)
- Risk metrics: Sharpe, Sortino, Alpha, Beta, Max Drawdown, VaR
- Benchmark comparison: rolling 3-month alpha vs Nifty 100
- Cohort analysis: investor tenure vs average SIP amount
- Geographic heatmap: T30 vs B30 city AUM contribution

### LAYER 5: VISUALISATION (Dashboard)

- Power BI / Tableau Desktop dashboard
- 4 report pages: Fund Overview, Performance, Investor, Industry Trends
- Slicers: Fund House, Category, Date Range, State
- KPI cards: AUM, SIP Inflow, Folio Count, Top Performer
- Export to PDF for stakeholder distribution

### 4.1 Database Schema (Star Schema)

| Table    | Type      | Key Columns                                         | Rows (approx.) |
|----------|-----------|-----------------------------------------------------|----------------|
| dim_fund | Dimension | amfi_code (PK), fund_house, category, expense_ratio | 40             |

|                                   |           |                                                                     |         |
|-----------------------------------|-----------|---------------------------------------------------------------------|---------|
| <a href="#">dim_date</a>          | Dimension | date_id (PK), date, year, month, quarter, is_weekday                | 1,500   |
| <a href="#">fact_nav</a>          | Fact      | amfi_code (FK), date (FK), nav, daily_return_pct                    | 46,000  |
| <a href="#">fact_transactions</a> | Fact      | tx_id (PK), investor_id, amfi_code (FK), date, amount, type         | 32,000+ |
| <a href="#">fact_performance</a>  | Fact      | amfi_code (FK), as_of_date, return_1yr, sharpe, alpha, max_drawdown |         |
| <a href="#">fact_portfolio</a>    | Fact      | amfi_code (FK), stock_symbol, weight_pct, sector, date              | 320     |
| <a href="#">fact_aum</a>          | Fact      | fund_house, date, aum_crore, num_schemes                            | 90      |
| <a href="#">fact_sip_industry</a> | Fact      | month, sip_inflow_crore, sip_accounts_crore                         | 48      |

## 5. 7-Day Task Breakdown

This project is designed to be completed **individually in 7 working days (~50–55 hours)**. Each day has a clear focus area, specific tasks, required tools, and concrete deliverables. Days 1-2 are foundational (data engineering), Days 3-4 are analytical, Days 5-6 are visualisation/advanced analytics, and Day 7 is documentation and presentation.

### DAY 1: Project Setup + Data Ingestion (ETL)

Week Day 1 | 6–8

#### Objectives

- Set up the project environment (Python, VS Code / Jupyter, Git)
- Understand the AMFI data ecosystem and public APIs
- Write Python scripts to ingest all 10 provided CSV datasets
- Fetch live NAV data from mfapi.in API for 5 selected schemes
- Create a Git repository and commit the initial project structure

| # | Task                                                                                                                                                | Tools / Tech           | Output / Deliverable                 | Time |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|--------------------------------------|------|
| 1 | Create project folder structure:<br>project/data/raw/data/processed/notebooks/<br>sql/dashboard/<br>reports/                                        | VS Code, Git           | Folder structure committed to GitHub | 30m  |
| 2 | Install dependencies: pip install pandas numpy matplotlib seaborn plotly sqlalchemy sqlite3 requests jupyter                                        | pip, requirements.txt  | requirements.txt file                | 20m  |
| 3 | Load all 10 CSV datasets using pandas, print shape/dtypes/head for each                                                                             | Python, Pandas         | data_ingestion.py script             | 60m  |
| 4 | Fetch live NAV from mfapi.in: GET https://api.mfapi.in/mf/125497 (HDFC Top 100) Parse JSON response and save as raw CSV                             | requests, json, Pandas | live_nav_fetch.py + raw CSV          | 45m  |
| 5 | Fetch NAV for 5 schemes: SBI Bluechip (119551), ICICI Bluechip (120503), Nippon Large Cap (118632), Axis Bluechip (119092), Kotak Bluechip (120841) | mfapi.in REST API      | 5 raw NAV CSV files                  | 60m  |
| 6 | Understand fund master: print unique fund houses, categories, sub-categories, risk grades                                                           | Pandas, Jupyter        | EDA cell outputs in notebook         | 30m  |
| 7 | Validate AMFI codes — check all codes in fund_master exist in nav_history                                                                           | Pandas merge/join      | Data quality report (text)           | 20m  |

| # | Task                                                      | Tools / Tech                                 | Output / Deliverable             | Time |
|---|-----------------------------------------------------------|----------------------------------------------|----------------------------------|------|
| 8 | Git commit with message: 'Day 1: Data ingestion complete' | <a href="#">Git</a> , <a href="#">GitHub</a> | <i>Remote repository updated</i> | 10m  |

**Day 1 Deliverables:** Project repo, `data_ingestion.py`, `live_nav_fetch.py`, 15 raw CSV files in `data/raw/`

## DAY 2: Data Cleaning + SQL Database Design

Week Day 2 | 7–8 hours

### Objectives

- Clean and validate all 10 datasets (handle nulls, duplicates, type errors)
- Design and implement a 5+ table SQLite database schema
- Load all cleaned data into the database
- Write and test 10 SQL queries for basic analytics

| # | Task                                                                                                                                                                                                                                                                                                                                                                                                                                    | Tools / Tech                                        | Output / Deliverable                   | Time |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|----------------------------------------|------|
| 1 | Clean <code>nav_history.csv</code> : - Parse dates to datetime - Sort by <code>amfi_code</code> + date - Forward-fill missing NAV (holidays) - Remove duplicates - Validate NAV > 0                                                                                                                                                                                                                                                     | <a href="#">Pandas</a> , <a href="#">datetime</a>   | <i>clean_nav.csv (46K rows)</i>        | 75m  |
| 2 | Clean <code>investor_transactions.csv</code> : - Standardise <code>transaction_type</code> (SIP/Lumpsum/Redemption) - Validate amount > 0 - Check KYC status values - Fix date formats                                                                                                                                                                                                                                                  | <a href="#">Pandas</a> , <a href="#">re</a>         | <i>clean_transactions.csv</i>          | 60m  |
| 3 | Clean <code>scheme_performance.csv</code> : - Validate return values are numeric - Flag negative Sharpe ratios - Check <code>expense_ratio</code> range (0.1% – 2.5%)                                                                                                                                                                                                                                                                   | <a href="#">Pandas</a> , <a href="#">numpy</a>      | <i>clean_performance.csv</i>           | 30m  |
| 4 | Design SQLite schema: CREATE TABLE <code>dim_fund</code> ( <code>amfi_code</code> TEXT PK, <code>fund_house</code> , <code>scheme_name</code> , <code>category</code> , ...) CREATE TABLE <code>fact_nav</code> ( <code>amfi_code</code> TEXT FK, <code>nav_date</code> DATE, <code>nav</code> REAL, <code>daily_return</code> REAL) CREATE TABLE <code>fact_transactions</code> (...) CREATE TABLE <code>fact_performance</code> (...) | <a href="#">SQLite</a> , <a href="#">SQL DDL</a>    | <i>schema.sql file</i>                 | 60m  |
| 5 | Load all cleaned datasets into SQLite: <code>engine = create_engine('sqlite:///bluestock_mf.db')</code> <code>df.to_sql('fact_nav', engine, ...)</code>                                                                                                                                                                                                                                                                                 | <a href="#">SQLAlchemy</a> , <a href="#">SQLite</a> | <i>bluestock_mf.db (database file)</i> | 45m  |

| # | Task                                                                                                                                                                         | Tools / Tech    | Output / Deliverable            | Time |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|---------------------------------|------|
| 6 | Write 10 SQL queries: 1. Top 5 funds by AUM 2. Average NAV per month 3. SIP inflow YoY growth 4. Transactions by state 5. Funds with expense_ratio < 1% 6-10: (see notebook) | SQL, SQLite CLI | <i>queries.sql with results</i> | 60m  |
| 7 | Create data dictionary documenting all columns, types, and sources                                                                                                           | Markdown/Excel  | <i>data_dictionary.md</i>       | 30m  |
| 8 | Git commit: 'Day 2: Cleaned data + SQLite DB loaded'                                                                                                                         | Git             | <i>GitHub updated</i>           | 10m  |

**Day 2 Deliverables: 10 clean CSVs, bluestock\_mf.db, schema.sql, queries.sql, data\_dictionary.md**

## DAY 3: Exploratory Data Analysis (EDA)

Week Day 3 | 7–8

### Objectives

- Perform deep EDA on NAV, AUM, SIP and investor data
- Create 15+ publication-quality charts using Matplotlib/Seaborn/Plotly
- Identify key trends, anomalies and insights
- Document findings in a structured Jupyter Notebook

| #  | Task                                                                                                                           | Tools / Tech                     | Output / Deliverable      | Time |
|----|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------|---------------------------|------|
| 1  | NAV trend analysis: Plot daily NAV for all 40 schemes 2022–2026. Highlight COVID recovery, 2023 rally, 2024 corrections.       | Matplotlib, Plotly               | Chart: NAV Trend Lines    | 60m  |
| 2  | AUM growth bar chart: Grouped bar chart — AUM by fund house for each year 2022–2025. Highlight SBI's dominance at Rs.12.5L Cr. | Seaborn barplot                  | Chart: AUM Growth by AMC  | 45m  |
| 3  | SIP inflow time-series: Monthly SIP inflow Jan 2022 to Dec 2025. Mark the Rs.31,002 Cr milestone (Dec 2025).                   | Plotly line chart                | Chart: SIP Inflow Trend   | 30m  |
| 4  | Category-wise inflow heatmap: Months on X-axis, categories on Y-axis, net inflow as colour intensity.                          | Seaborn heatmap                  | Chart: Category Heatmap   | 45m  |
| 5  | Investor demographics: Age group distribution pie chart. SIP amount box plot by age group.                                     | Matplotlib, Seaborn              | 2 Charts: Demographics    | 45m  |
| 6  | Geographic distribution: Horizontal bar chart — SIP amount by state. T30 vs B30 pie chart.                                     | Matplotlib, geopandas (optional) | Chart: Geo Distribution   | 60m  |
| 7  | Folio count growth: Line chart Jan 2022 to Dec 2025 showing growth from 13.26 to 26.12 crore.                                  | Plotly                           | Chart: Folio Count Growth | 30m  |
| 8  | Correlation matrix: Compute pairwise correlation of NAV returns across 10 selected funds.                                      | Seaborn heatmap, numpy           | Chart: Correlation Matrix | 45m  |
| 9  | Top holdings sector distribution: Pie/donut chart of sector weights across all equity fund portfolios.                         | Matplotlib donut                 | Chart: Sector Allocation  | 30m  |
| 10 | Summarise 10 key EDA findings as bullet points in notebook markdown cells                                                      | Jupyter Markdown                 | EDA_Findings.ipynb        | 30m  |

**Day 3 Deliverables:** EDA\_Analysis.ipynb with 15+ charts, EDA\_Findings summary, exported PNG charts

## DAY 4: Fund Performance Analytics

Week Day 4 | 7-8 hours

### Objectives

- Compute all key performance and risk metrics from NAV history
- Build a fund ranking/scoring model
- Compare fund returns against benchmark indices
- Identify best and worst performing funds per category

| # | Task                                                                                                                                                                                                                                                             | Tools / Tech              | Output / Deliverable | Time |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|----------------------|------|
| 1 | Compute daily returns for all funds:<br>$\text{daily\_return} = \text{nav\_t} / \text{nav\_t-1} - 1$<br>Annualised return = $(1 + \text{daily\_return}).\text{prod}()^{(252/n)} - 1$                                                                             | Pandas, numpy             | returns_computed.csv | 45m  |
| 2 | Calculate CAGR for 1yr, 3yr, 5yr periods: $\text{CAGR} = (\text{NAV\_end} / \text{NAV\_start})^{(1/n)} - 1$ for SBI Bluechip, HDFC Top 100, etc.                                                                                                                 | Pandas groupby, numpy     | cagr_report.csv      | 60m  |
| 3 | Compute Sharpe Ratio: $\text{Sharpe} = (\text{Rp} - \text{Rf}) / \text{Std}(\text{Rp})$ Use $\text{Rf} = 6.5\%$ (RBI repo rate proxy) Annualise with $\text{sqrt}(252)$                                                                                          | numpy, Pandas             | sharpe_values.csv    | 45m  |
| 4 | Compute Sortino Ratio: $\text{Sortino} = (\text{Rp} - \text{Rf}) / \text{Downside\_Std}$ where Downside_Std uses only negative return days                                                                                                                       | numpy                     | sortino_values.csv   | 30m  |
| 5 | Compute Alpha & Beta vs benchmark: Regress fund returns on Nifty 100 returns (OLS) Alpha = intercept * 252, Beta = slope Use <code>scipy.stats.linregress</code>                                                                                                 | scipy, numpy              | alpha_beta.csv       | 60m  |
| 6 | Compute Maximum Drawdown: $\text{max\_dd} = \min(\text{NAV} / \text{running\_max} - 1)$ Highlight worst drawdown period for each fund                                                                                                                            | Pandas cummax, numpy      | max_drawdown.csv     | 45m  |
| 7 | Build Fund Scorecard (composite score 0-100): Score = $30\% \times (\text{3yr return rank}) + 25\% \times (\text{Sharpe rank}) + 20\% \times (\text{Alpha rank}) + 15\% \times (\text{Expense ratio rank, inverse}) + 10\% \times (\text{Max DD rank, inverse})$ | Pandas rank, weighted sum | fund_scorecard.csv   | 60m  |
| 8 | Benchmark comparison chart: Plot top 5 funds vs Nifty 50 and Nifty 100 over 3 years. Compute tracking error.                                                                                                                                                     | Matplotlib, numpy std     | benchmark_chart.png  | 45m  |

**Day 4 Deliverables:** Performance\_Analytics.ipynb, fund\_scorecard.csv, alpha\_beta.csv, benchmark comparison chart

## DAY 5: Dashboard Development (Power BI/ Tableau)

Week Day 5 | 7–8

### Objectives

- Connect Power BI / Tableau to the SQLite database and cleaned CSVs
- Build 4 interactive dashboard pages
- Add slicers, drill-through, and tooltips
- Publish / export the dashboard

| # | Task                                                                                                                                                                                                                    | Tools / Tech        | Output / Deliverable   | Time |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|------------------------|------|
| 1 | Connect Power BI to bluestock_mf.db (SQLite ODBC) OR import CSVs directly. Verify all 8 tables load correctly.                                                                                                          | Power BI / Tableau  | Data model loaded      | 45m  |
| 2 | Build PAGE 1 — Industry Overview: KPI Cards: Total AUM (Rs.81L Cr), SIP Inflows (Rs.31K Cr), Folios (26.12 Cr), # Schemes (1908) Line chart: Industry AUM Jan2022-Dec2025 Bar chart: AUM by fund house (top 10)         | Power BI            | Dashboard Page 1       | 75m  |
| 3 | Build PAGE 2 — Fund Performance: Scatter plot: Return (X) vs Risk/StdDev (Y), bubble=AUM Table: Sortable fund scorecard Line chart: NAV of selected fund vs benchmark Slicer: Fund House, Category, Plan                | Power BI            | Dashboard Page 2       | 75m  |
| 4 | Build PAGE 3 — Investor Analytics: Map/Bar: Transaction amount by state Donut: SIP vs Lumpsum vs Redemption split Bar: Age group vs avg SIP amount Line: Monthly transaction volume Slicer: State, Age Group, City Tier | Power BI            | Dashboard Page 3       | 75m  |
| 5 | Build PAGE 4 — SIP & Market Trends: Dual-axis: SIP Inflow (bar) + Nifty 50 (line) 2022-2025 Heat map: Category inflows by month Bar: Top 5 categories by net inflow FY25 KPI: SIP accounts growth YoY                   | Power BI            | Dashboard Page 4       | 60m  |
| 6 | Add tooltips, drill-through from fund table to NAV chart. Add Bluestock logo and colour theme.                                                                                                                          | Power BI formatting | Branded dashboard      | 30m  |
| 7 | Export dashboard to PDF. Export each page as PNG for the final report.                                                                                                                                                  | Power BI Export     | Dashboard.pdf + 4 PNGs | 20m  |

**Day 5 Deliverables:** bluestock\_mf\_dashboard.pbix (Power BI file), Dashboard.pdf, 4 page screenshots

## DAY 6: Advanced Analytics + Risk Metrics

Week Day 6 | 6–7

### Objectives

- Implement Value at Risk (VaR) and Conditional VaR
- Perform investor cohort analysis
- Build a simple fund recommendation model
- Analyse sector concentration risk in portfolios

| # | Task                                                                                                                                                                                  | Tools / Tech                                                 | Output / Deliverable                  | Time |
|---|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|---------------------------------------|------|
| 1 | Compute Historical VaR (95%) for each fund: VaR = 5th percentile of daily return distribution CVaR = mean of returns below VaR threshold                                              | <code>numpy percentile,</code><br><code>Pandas</code>        | <code>var_cvar_report.csv</code>      | 60m  |
| 2 | Compute Rolling 90-day Sharpe Ratio for 5 funds: <code>rolling_sharpe = returns.rolling(90).mean() / returns.rolling(90).std() * sqrt(252)</code><br>Plot over time                   | <code>Pandas rolling,</code><br><code>Matplotlib</code>      | <code>rolling_sharpe_chart.png</code> | 45m  |
| 3 | Investor cohort analysis: Group investors by first transaction year (2024/2025) Compute average SIP amount, total invested, fund preference by cohort                                 | <code>Pandas groupby,</code><br><code>pivot</code>           | <code>cohort_analysis.csv</code>      | 60m  |
| 4 | SIP continuation analysis: For each investor with 6+ SIP transactions, compute average gap between transactions. Flag investors with gaps > 35 days as 'at-risk'                      | <code>Pandas date diff,</code><br><code>groupby</code>       | <code>sip_continuity.csv</code>       | 45m  |
| 5 | Simple fund recommendation logic: Input: investor risk appetite (Low/Moderate/High) Output: Top 3 funds by Sharpe ratio within matching risk_grade Print recommendation table         | <code>Python functions,</code><br><code>Pandas filter</code> | <code>recommender.py</code>           | 45m  |
| 6 | Sector concentration analysis: For each equity fund, compute Herfindahl-Hirschman Index (HHI) of sector weights. $HHI = \sum(\text{weight}_i^2)$ . High HHI = concentrated portfolio. | <code>numpy, Pandas</code><br><code>groupby</code>           | <code>sector_hhi.csv + chart</code>   | 45m  |
| 7 | Write advanced analytics summary with 5 key insights (e.g. which funds have highest VaR, which cohorts invest most, etc.)                                                             | <code>Jupyter Markdown</code>                                | <code>Advanced_Analytics.ipynb</code> | 30m  |

**Day 6 Deliverables:** `Advanced_Analytics.ipynb`, `var_cvar_report.csv`, `rolling_sharpe_chart.png`, `recommender.py`

## DAY 7: Final Report + Presentation + Deployment

Week Day 7 | 6–7 hours

### Objectives

- Write the comprehensive final project report (PDF)
- Create a 12-slide presentation deck
- Clean and document all code (README, docstrings)
- Push final code to GitHub with proper README
- Optional: Deploy dashboard to Power BI Service / Tableau Public

| # | Task                                                                                                                                                                                                                                                             | Tools / Tech                      | Output / Deliverable           | Time |
|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|--------------------------------|------|
| 1 | Write Final Report (this document serves as template): Sections: Executive Summary, Data, ETL, EDA Findings, Performance Analysis, Dashboard Screenshots, Recommendations, Limitations                                                                           | Word / LaTeX / Python ReportLab   | Final_Report.pdf (15–20 pages) | 120m |
| 2 | Create 12-slide presentation deck: Slide 1: Title Slide 2: Problem & Objective Slide 3: Data Sources Slide 4: Architecture Slide 5-6: EDA Highlights Slide 7-8: Performance Metrics Slide 9-10: Dashboard Screenshots Slide 11: Key Findings Slide 12: Thank You | PowerPoint / Google Slides        | Bluestock_MF_Presentation.pptx | 90m  |
| 3 | Clean all Python scripts: Add docstrings, remove debug prints Add README.md to each folder Create master run script: python run_pipeline.py                                                                                                                      | VS Code, Python                   | Clean codebase                 | 45m  |
| 4 | Write root README.md: Project overview, setup instructions, file descriptions, how to run ETL, how to open dashboard                                                                                                                                             | Markdown, Git                     | README.md                      | 30m  |
| 5 | Final GitHub push: git add . git commit -m 'Final: Complete Bluestock MF Capstone' git push origin main Tag release: git tag v1.0                                                                                                                                | Git, GitHub                       | Public GitHub repo             | 20m  |
| 6 | Optional: Publish Power BI dashboard to Power BI Service (free account). OR publish to Tableau Public: File → Publish to Tableau Public                                                                                                                          | Power BI Service / Tableau Public | Public dashboard URL           | 30m  |
| 7 | Self-review checklist: ✓ All 8 objectives met? ✓ All 7 deliverables submitted? ✓ Code runs without errors? ✓ Dashboard loads correctly? ✓ Report is professional?                                                                                                | Checklist                         | Submission-ready package       | 15m  |

**Day 7 Deliverables: Final\_Report.pdf, Presentation.pptx, clean GitHub repo with README, dashboard published**

## 6. Technical Stack Details

| Category          | Tool / Library                    | Version  | Purpose                             |
|-------------------|-----------------------------------|----------|-------------------------------------|
| Language          | <a href="#">Python</a>            | 3.10+    | All data processing, ETL, analytics |
| Data Manipulation | <a href="#">Pandas</a>            | 2.0+     | DataFrames, cleaning, aggregation   |
| Numerical         | <a href="#">NumPy</a>             | 1.24+    | Risk metrics, statistical functions |
| Visualisation     | <a href="#">Matplotlib</a>        | 3.7+     | Static charts for reports           |
| Visualisation     | <a href="#">Seaborn</a>           | 0.12+    | Heatmaps, distributions             |
| Visualisation     | <a href="#">Plotly</a>            | 5.x      | Interactive charts in Jupyter       |
| Database          | <a href="#">SQLite3</a>           | Built-in | Local relational database           |
| ORM               | <a href="#">SQLAlchemy</a>        | 2.0      | Python-to-SQLite interface          |
| Statistics        | <a href="#">SciPy</a>             | 1.10+    | OLS regression for Beta/Alpha       |
| Notebooks         | <a href="#">Jupyter Lab</a>       | 4.x      | EDA and analytics notebooks         |
| Dashboard         | <a href="#">Power BI Desktop</a>  | Latest   | Interactive BI dashboard            |
| Dashboard (alt)   | <a href="#">Tableau Desktop</a>   | 2024.x   | Alternative to Power BI             |
| Version Control   | <a href="#">Git + GitHub</a>      | Latest   | Code versioning, submission         |
| API               | <a href="#">mfapi.in</a>          | v1       | Live NAV data (no auth required)    |
| API (alt)         | <a href="#">mfdata.in</a>         | v1       | NAV + expense ratio data            |
| HTTP Client       | <a href="#">Requests</a>          | 2.30+    | API calls from Python               |
| IDE               | <a href="#">VS Code / PyCharm</a> | Latest   | Code development                    |
| Reporting         | <a href="#">ReportLab / Word</a>  | 4.x      | Final PDF report                    |

### 6.1 Folder Structure

```

bluestock_mf_capstone/ data/ raw/ ← Original downloaded files processed/
← Cleaned, merged CSVs db/ ← bluestock_mf.db (SQLite)notebooks/
01_data_ingestion.ipynb 02_data_cleaning.ipynb 03_eda_analysis.ipynb
04_performance_analytics.ipynb 05_advanced_analytics.ipynb scripts/ etl_pipeline.py
← Master ETL script live_nav_fetch.py ← mfapi.in fetcher compute_metrics.py ←
Performance metrics recommender.py ← Fund recommendersql/
schema.sql ← CREATE TABLE statements queries.sql ← 10 analytical
queriesdashboard/ bluestock_mf.pbix ← Power BI dashboardreports/ Final_Report.pdf
Presentation.pptxREADME.md

```

## 7. Deliverables & Evaluation Rubric

| #  | Deliverable           | Format         | Weight | Evaluation Criteria                                     |
|----|-----------------------|----------------|--------|---------------------------------------------------------|
| D1 | ETL Pipeline Script   | Python .py     | 15%    | Code quality, handles errors, runs without manual steps |
| D2 | SQLite Database       | .db file       | 10%    | Correct schema, all data loaded, queries run correctly  |
| D3 | EDA Notebook          | .ipynb         | 15%    | Depth of analysis, chart quality, insights documented   |
| D4 | Performance Metrics   | .ipynb + CSV   | 15%    | Mathematical accuracy, correct use of risk formulas     |
| D5 | Interactive Dashboard | .pbix or .twbx | 20%    | Design quality, functionality, all 4 pages complete     |
| D6 | Advanced Analytics    | .ipynb         | 10%    | VaR correctness, cohort analysis insight quality        |
| D7 | Final Report + Slides | .pdf + .pptx   | 15%    | Professional quality, completeness, clarity of findings |

### 7.1 Bonus Challenges (Optional +10 marks each)

- Deploy ETL pipeline as a scheduled script that auto-fetches NAV from mfapi.in every weekday at 8 PM
- Build a simple Streamlit web app as an alternative to Power BI for the dashboard
- Implement a Monte Carlo simulation to project NAV growth over 5 years with uncertainty bands
- Add a portfolio optimisation module (Markowitz Efficient Frontier) for 5 selected funds
- Create an automated email report generator that sends weekly performance summary as HTML

### 7.2 Common Mistakes to Avoid

| Mistake                                                            | Correct Approach                                                        |
|--------------------------------------------------------------------|-------------------------------------------------------------------------|
| ■ Using simulated random NAV data without anchoring to real values | ■ Use the real AMFI NAV anchor values provided in the dataset generator |
| ■ Hard-coding file paths                                           | ■ Use pathlib.Path or os.path.join for cross-platform compatibility     |
| ■ Not handling weekends/holidays in NAV data                       | ■ Use forward-fill (ffill) after reindexing to full date range          |
| ■ Computing CAGR with calendar days instead of trading days        | ■ Count actual rows (trading days) or use (252/n_days) annualisation    |
| ■ Dashboard with no filters/slicers                                | ■ Every page must have at least 2 interactive slicers                   |
| ■ Confusing AUM (lakh crore) with scheme-level AUM (crore)         | ■ Always include units in column names: aum_lakh_crore vs aum_crore     |
| ■ Uploading .db files >100MB to GitHub                             | ■ Add *.db to .gitignore; upload schema.sql + instructions instead      |

## 8. Appendix: Dataset Schema Reference

### 01\_fund\_master.csv

| Column             | Type | Description                                                 |
|--------------------|------|-------------------------------------------------------------|
| amfi_code          | TEXT | AMFI unique scheme code (e.g. 125497 = HDFC Top 100 Direct) |
| fund_house         | TEXT | AMC name (e.g. SBI Mutual Fund, HDFC Mutual Fund)           |
| scheme_name        | TEXT | Full official AMFI scheme name                              |
| category           | TEXT | Equity / Debt / Hybrid                                      |
| sub_category       | TEXT | Large Cap / Mid Cap / Small Cap / Liquid / etc.             |
| plan               | TEXT | Regular or Direct                                           |
| launch_date        | DATE | Fund launch date (YYYY-MM-DD)                               |
| benchmark          | TEXT | Official benchmark index                                    |
| expense_ratio_pct  | REAL | Annual expense ratio in % (e.g. 1.05)                       |
| exit_load_pct      | REAL | Exit load % (0 for Liquid/Index funds)                      |
| fund_manager       | TEXT | Name of primary fund manager                                |
| risk_category      | TEXT | SEBI risk category: Low / Moderate / High / Very High       |
| sebi_category_code | TEXT | Internal code: EC01=LargeCap, EC03=SmallCap, DC01=Liquid    |

### 02\_nav\_history.csv

| Column    | Type | Description                                                   |
|-----------|------|---------------------------------------------------------------|
| amfi_code | TEXT | Foreign key to fund_master                                    |
| date      | DATE | NAV date (business days only)                                 |
| nav       | REAL | NAV in Rs. (e.g. 892.4560). Anchored to real mfapi.in values. |

### 04\_monthly\_sip\_inflows.csv

| Column                  | Type | Description                                           |
|-------------------------|------|-------------------------------------------------------|
| month                   | TEXT | YYYY-MM format                                        |
| sip_inflow_crore        | REAL | Total SIP inflows in Rs. crore (real AMFI data)       |
| active_sip_accounts_cro | REAL | Number of actively contributing SIP accounts in crore |
| new_sip_accounts_lakh   | REAL | New SIP registrations in that month (lakh accounts)   |
| sip_aum_lakh_crore      | REAL | Total SIP assets under management in Rs. lakh crore   |
| yoy_growth_pct          | REAL | YoY growth % in SIP inflows (computed)                |

### 07\_scheme\_performance.csv

| Column | Type | Description |
|--------|------|-------------|
|--------|------|-------------|

|                    |      |                                                     |
|--------------------|------|-----------------------------------------------------|
| return_1yr_pct     | REAL | 1-year absolute return %                            |
| return_3yr_pct     | REAL | 3-year CAGR %                                       |
| return_5yr_pct     | REAL | 5-year CAGR %                                       |
| benchmark_3yr_pct  | REAL | Benchmark index 3yr CAGR for comparison             |
| alpha              | REAL | Return above benchmark (return_3yr - benchmark_3yr) |
| beta               | REAL | Sensitivity to market (1.0 = same as market)        |
| sharpe_ratio       | REAL | Risk-adjusted return (higher is better, >1 is good) |
| sortino_ratio      | REAL | Like Sharpe but penalises only downside volatility  |
| std_dev_ann_pct    | REAL | Annualised standard deviation of daily returns (%)  |
| max_drawdown_pct   | REAL | Worst peak-to-trough decline (negative value)       |
| morningstar_rating | INT  | 1-5 star rating (simulated, based on Sharpe)        |

## 08\_investor\_transactions.csv

| Column             | Type | Description                                                        |
|--------------------|------|--------------------------------------------------------------------|
| investor_id        | TEXT | Unique investor identifier (INV000001 to INV005000)                |
| transaction_date   | DATE | Date of SIP/Lumpsum/Redemption                                     |
| amfi_code          | TEXT | Fund in which transaction occurred                                 |
| transaction_type   | TEXT | SIP / Lumpsum / Redemption                                         |
| amount_inr         | INT  | Transaction amount in Indian Rupees                                |
| state              | TEXT | Investor's state (12 Indian states covered)                        |
| city               | TEXT | Investor's city                                                    |
| city_tier          | TEXT | T30 (Top 30 cities) or B30 (Beyond Top 30) per AMFI classification |
| age_group          | TEXT | 18-25 / 26-35 / 36-45 / 46-55 / 56+                                |
| gender             | TEXT | Male / Female                                                      |
| annual_income_lakh | REAL | Annual income in Rs. lakh                                          |
| payment_mode       | TEXT | UPI / Net Banking / Mandate / Cheque                               |
| kyc_status         | TEXT | Verified (92%) / Pending (8%)                                      |